

WEF1VO Web Fundamentals

Vorlesungsunterlagen

Wolfgang Hochleitner
wolfgang.hochleitner@fh-hagenberg.at

WS 2025/26

Vorlesung 1

Einführung

1.1 Zur Web Fundamentals Vorlesung

Die Vorlesung Web Fundamentals – kurz WEF1VO – findet im ersten Semester des Bachelorstudiums Medientechnik und -design statt. Im zweiten Semester wird sie durch Web Technologies (WET2VO), im dritten durch Web Architectures (WAR3VO) fortgesetzt. Weiters ist diese Vorlesung inhaltlich mit Web Design (WED1IL) abgestimmt, welches die konzeptionellen Grundlagen für die hier vermittelten technischen Inhalte liefert. Im Folgenden sind die für dieses Semester relevanten Informationen kurz zusammengefasst.

1.1.1 Organisatorisches

Die Vorlesung zu Web Fundamentals behandelt die Grundlagen der Webentwicklung und ist auf *zwölf Termine zu je zwei Einheiten* aufgeteilt. Die Beurteilung erfolgt im Rahmen einer *elektronischen Klausur*, die am Ende des Semesters stattfindet. Die Klausur dauert 90 Minuten und findet ohne Zuhilfenahme der Unterlagen statt.

Die dazugehörigen Übungen sind auf *zwölf Termine zu zwei Einheiten* aufgeteilt. In jeder Übung werden die Inhalte der Vorlesung praktisch angewandt. Die Beurteilung der Übungen erfolgt durch fünf Übungsabgaben, die zu ausgewählten Übungen einzureichen sind. Die restlichen Übungen finden als code-along Einheiten statt und dienen der gemeinsamen Erarbeitung von Inhalten.

Die Unterlagen zum Kurs befinden sich auf der eLearning-Plattform der FH Oberösterreich in den zugehörigen Kursen für Vorlesung¹ und Übung².

1.1.2 Inhalte der Vorlesung

Im Verlauf des Semesters werden folgende Themengebiete behandelt:

- Hypertext Markup Language (HTML),
- Cascading Style Sheets (CSS),
- Box-Modell, Responsive Web und Layout-Frameworks,

¹<https://elearning.fh-ooe.at/course/view.php?id=38355>

²<https://elearning.fh-ooe.at/course/view.php?id=38353>

- Web Accessibility,
- Datenformate (XML, YAML, JSON),
- JavaScript/ECMAScript,
- JavaScript DOM,
- JavaScript Workflow & Tools,
- HTML JavaScript APIs.

1.2 Begriffsdefinitionen

Bevor mit der Vermittlung von Inhalten begonnen werden kann, ist es notwendig, die wichtigsten Begriffe aus dem Fachbereich zu definieren bzw. abzuklären.

1.2.1 Hypermedia

Hypermedia ist die Erweiterung von Hypertext durch die Einbindung weiterer Medien wie z. B. Bild, Ton oder Bewegtbild. Das größte, derzeit existierende Hypermedia-System ist das World Wide Web.

1.2.2 Hypertext

Als *Hypertext* wird eine Menge von Dokumenten (Knoten) bezeichnet, die Querverweise (Links, Kanten) enthalten, welche es den Leser*innen erlauben, mithilfe einer interaktiven Software (Browser) einfach von einem Dokument zum anderen zu gelangen. Der Begriff wurde vom US-amerikanischen Soziologen Ted Nelson 1963 zum ersten Mal verwendet. In der 1987 veröffentlichten Ausgabe von [5] definiert er den Begriff folgendermaßen:

By Hypertext I simply mean non-sequential writing; a body of written or pictorial material interconnected in such a complex way that it could not be presented or represented on paper. Hypertext is the generic term for any text, which cannot be printed.

1.2.3 World Wide Web

Das *World Wide Web* (auch WWW oder einfach Web) ist ein globales, interaktives Informationssystem, das auf Hypertext bzw. Hypermedia basiert. Entgegen der Verwendung im allgemeinen Sprachgebrauch ist das World Wide Web nicht mit dem Internet gleichzusetzen. Das World Wide Web ist lediglich einer von vielen Diensten im Internet. Andere sind z. B. E-Mail, Instant Messaging, Streamingdienste oder Cloud-Services. Das Internet ist also das Computernetz, über welches das World Wide Web verfügbar gemacht wird.

1.2.4 HTML

Die *Hypertext Markup Language* (HTML) ist das Dokumentenformat des WWW. HTML ist eine textbasierte Auszeichnungssprache (engl. „markup language“). Das Markup verleiht dem Text eine Struktur, die von Software (z. B. Webbrowsern) gelesen werden kann.

1.2.5 URL und URI

Ein URL – kurz für *Uniform Resource Locator* – ist ein Identifikationsschema, mit dem eine Ressource (z. B. eine Website) im Internet eindeutig zugeordnet werden kann. Ein URL ist (vereinfacht dargestellt) aus der Zugriffsmethode (meist ein Netzwerkprotokoll wie HTTP) und einem für diese Zugriffsmethode spezifischen Teil (meist Domain-Name und gesuchte Datei) aufgebaut: `http://www.domain.com/file.html`. URLs sind in RFC 1738 [2] – einem Standard der Internet Engineering Taskforce (IETF)³ – definiert.

Ein URL ist genau genommen eine Unterform eines URIs, des *Uniform Resource Identifiers*. Diese Art der Bezeichnung von Ressourcen ist noch etwas allgemeiner und umfassender und wurde im RFC 3986 [1] formuliert.

Um URLs zu standardisieren, existiert auch ein URL Living Standard [9] der WHAT-WG (siehe Abschnitt 1.7), der versucht, die IETF-Standards zu vereinheitlichen und mit den Gegebenheiten moderner Browser abzugleichen.

1.2.6 HTTP

Das *Hypertext Transfer Protokoll* (kurz HTTP) ist das Übertragungsprotokoll für Daten über ein Netzwerk, im Speziellen für Websites. Details zu HTTP werden im zweiten Semester in Web Technologies (WET2VO) behandelt.

Wichtig: Im Akronym HTTP ist das Wort „Protokoll“ bereits enthalten. Von einem „HTTP-Protokoll“ zu sprechen ist daher eine Tautologie und sollte vermieden werden.

1.2.7 Website vs. Webpage vs. Homepage

Im allgemeinen Sprachgebrauch werden die Begriffe Website, Webpage und Homepage oft gleichgesetzt bzw. schlichtweg falsch verwendet. Jedoch handelt es sich um unterschiedliche Dinge: Eine *Website* ist ein Verbund mehrerer, einzelner *Webpages*. Eine dieser Webpages ist wiederum die *Homepage*, die als Start- oder Einstiegsseite in die Website fungiert. Abbildung 1.1 stellt dies schematisch dar.

1.3 Funktionsweise des World Wide Web

Beim Aufruf einer Website findet ein Kommunikationsprozess statt, in den zwei Parteien involviert sind. Der *Client* fordert eine Website an, diese wird von einem *Server* auf der Gegenstelle bereitgestellt und zurück geliefert. Die Website selbst liegt im Dateisystem des Servers. Die Kommunikation erfolgt dabei über das Internet mit Hilfe von HTTP (was dabei genau passiert, in WET2VO erläutert). Dieser Ablauf ist in Abbildung 1.2 dargestellt.

Im Falle von einfachen, statischen HTML-Seiten kann der Server auch entfallen und diese können direkt lokal vom eigenen Rechner geöffnet werden. Erst bei serverseitigen Technologien (z. B. PHP) ist ein Webserver zwingend nötig. Notwendig ist dieser natürlich auch dann, wenn HTML-Seiten der Öffentlichkeit über das WWW zugänglich gemacht werden sollen (im Gegensatz zu lokalem Entwickeln, wie es im Rahmen der WEF-Übungen geschieht).

³<https://www.ietf.org/>



Abbildung 1.1: Eine Website enthält mehrere einzelne Webpages. Eine dieser Webpages ist die Homepage, also die Startseite der Website.

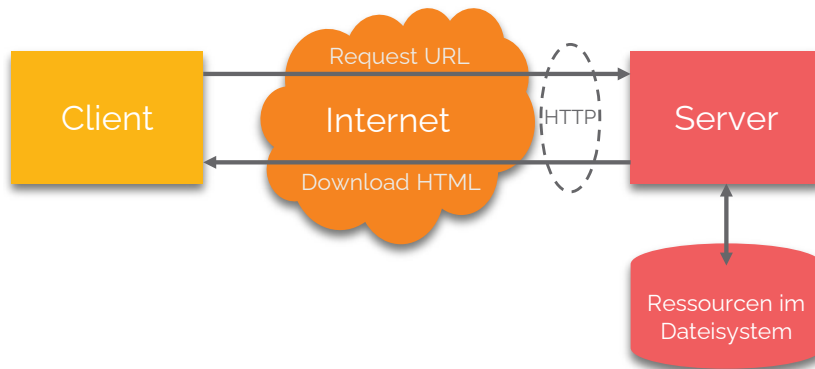


Abbildung 1.2: Client-Server-Kommunikation über das Internet.

Der typische Vertreter eines Clients ist der *Webbrowser*, aber auch eine *Suchmaschine* ist eine Art von Client. Auch sie ruft Websites ab, nur werden deren Inhalte indiziert und nicht angezeigt, wie es beim Webbrowser üblicherweise der Fall ist.

1.4 Webbrowser

Der *Webbrowser*, kurz *Browser* oder auch *Web-Client* genannt, ist die Software, die im Kommunikationsmodell die Anfragen an den Server stellt. Er ist auch gleichzeitig die Visualisierungsoberfläche für HTML-Markup, stellt also dieses Markup grafisch dar. Dieser Vorgang wird auch das Rendern des HTML-Markups genannt. Verantwortlich für diesen Vorgang ist die Rendering-Engine, die quasi unter der Haube des Browsers steckt. Sie ist auch dafür verantwortlich, wie gut ein Browser diverse Webstandards erfüllt, wie dieser Scriptsprachen wie JavaScript interpretiert und in welcher Form neue Technologien unterstützt werden. Es gibt derzeit fünf Browser, die im Bereich der Webentwicklung am Desktop eine Rolle spielen.

1.4.1 Google Chrome

*Chrome*⁴ wird von Google entwickelt und ist seit Ende 2008 als freier Browser verfügbar. Die aktuelle Version ist 141.0. Dies belegt den sehr kurzen Releasezyklus. Chrome ist als Open-Source Projekt unter dem Namen Chromium verfügbar, welches vor allem bei Linux-Distributionen zum Einsatz kommt.

Neu war zum Einführungszeitpunkt vor allem, dass in Chrome alle Tabs als eigene Prozesse realisiert (d. h. eigene kleine Programmteile) sind. Bei einem Absturz eines Tabs wird nicht der gesamte Browser in Mitleidenschaft gezogen. Auch hat dieses „abgeschlossene“ Tabbed Browsing Sicherheitsvorteile (Schadcode bleibt im einzelnen Tab und kann nicht übergreifen).

Chrome verwendete bis Version 27.0 die *WebKit2*-Engine. Diese basiert auf der unter KDE (Linux) entwickelten *KHTML*-Engine und wird nach wie vor aktiv von Apple weiter entwickelt. Die WebKit-Engine gehört auch zu den performantesten Renderern, sowohl für HTML als auch bei der Interpretierung von JavaScript. Seit Version 28.0 kommt in Chrome eine Abspaltung (engl. „fork“) von WebKit namens *Blink* zum Einsatz. Als Grund für den Umstieg wird eine größere Flexibilität bei der Entwicklung von Chrome-spezifischen Features begründet. Während früher WebKit als führend bei der Implementierung von Standards und der Integration neuer Technologien angesehen wurde, kommt diese Rolle nun eher Blink zu. Laut [7] liegt der Marktanteil für Oktober 2025 im Desktop-Bereich bei 79,53 %, was ihn zum meistbenutzten Browser macht.

1.4.2 Microsoft Edge

*Microsoft Edge*⁵ ist der aktuelle Browser von Microsoft. Er wurde erstmals mit Windows 10 veröffentlicht und legt den Fokus stark auf die Unterstützung von aktuellen und zukünftigen Web-Technologien, alte und proprietäre Technologien (wie etwa ActiveX) werden nicht mehr unterstützt. Als Engine kam mit der Veröffentlichung 2015 EdgeHTML (eine Abspaltung der Internet Explorer Engine Trident) zum Einsatz. 2018 kündigte Microsoft jedoch an, die Entwicklung der eigenen Engine einzustellen, und Edge fortan auf Blink zu basieren. Seit Anfang 2020 ersetzen diese Versionen die vorigen (nun Edge Legacy genannten) Versionen vollständig. Die Versionsnummern sind denen der Blink-Engine angepasst, die aktuelle ist daher wie bei Google Chrome 141.0. Der Marktanteil von Edge liegt laut [7] im Oktober 2025 bei 8,01 %, was ihn zur Nummer zwei am Desktop-Browsermarkt macht.

Der Vorgänger von Edge war der allseits bekannte *Internet Explorer*, der von 1995 bis 2015 entwickelt wurde. Die letzte Version war 11.0. Internet Explorer war lange Zeit der marktbeherrschende Browser bzw. kämpfte mit dem Netscape Navigator im sogenannten Browserkrieg⁶ (1995–1998) um die Vorherrschaft. In diese Zeit fällt die Entwicklung immer neuer Funktionalitäten, die auf Kosten der Webstandards gingen und für eine Vielzahl an noch immer vorherrschenden Kompatibilitätsproblemen im Web verantwortlich sind. Internet Explorer verwendete seit Version 4.0 die *Trident*-Engine. Die Engine stand lange in der Kritik, Webstandards nur unzureichend zu unterstützen bzw. falsch

⁴<https://www.google.com/chrome/>

⁵<https://www.microsoft.com/edge/>

⁶<https://de.wikipedia.org/wiki/Browserkrieg>

umzusetzen, wenngleich die letzte Version zufriedenstellender war. Allerdings wird diese nicht mehr weiterentwickelt, weshalb neue Technologien nicht mehr unterstützt werden. Laut [7] belegt der Internet Explorer mit einem Anteil von 0,18 % im Oktober 2025 mittlerweile keine Rolle mehr unter den Desktop-Browsern.

1.4.3 Apple Safari

*Safari*⁷ ist ein Browser, der in Apples Betriebssystem macOS integriert ist. Die aktuelle Version ist 26.0. Im September wurde die Versionierung der von iOS und iPadOS angepasst und ist somit das Kalenderjahr, welches der initialen Veröffentlichung folgt. Kurzzeitig existierten auch Versionen für Microsoft Windows, jedoch wurde Safari ab Version 6 nicht mehr für Windows veröffentlicht.

Wie früher Chrome verwendet auch Safari die *WebKit2*-Engine, es gelten zum Großteil die selben Dinge, die bereits in Abschnitt 1.4.1 erwähnt wurden, wenngleich Apple eine eigene Abspaltung der WebKit-Engine verwendet. Der Marktanteil im Desktop-Bereich laut [7] im Oktober 2025 beträgt 4,35 %. Damit ist der Browser die aktuelle Nummer drei am Desktop-Browsermarkt.

1.4.4 Mozilla Firefox

*Firefox*⁸ ist ein freier Webbrowser der Mozilla Foundation und seit 2002 in Entwicklung. Er gilt als Nachfolger des Netscape Navigators, einer der ersten populären Browser (neben dem Microsoft Internet Explorer). Die aktuelle Version ist 144.0. Der Releasezyklus neuer Versionen beträgt ca. 6 Wochen.

Mozilla Firefox verwendet intern die hauseigene *Gecko*-Engine. Diese zielt darauf ab, möglichst viele Webstandards umzusetzen. Bei der Umsetzung kommender Standards bzw. neuer Entwicklungen ist Gecko gut vertreten, jedoch nicht immer an vorderster Front. Aus diesem Grund wurde 2013 begonnen, eine neue Engine namens *Servo* von Grund auf zu entwickeln. Da diese jedoch in nicht absehbarer Zeit einsatzbereit war, wurde das Quantum Projekt gestartet, das fertige Servo-Komponenten in Gecko integrieren soll. Seit Version 57 hieß Firefox daher *Firefox Quantum*, da die ersten Servo-Komponenten bereits integriert wurden. Im Oktober 2019 wurde er jedoch wieder auf „Firefox Browser“ umbenannt. Firefox hatte im Oktober 2025 laut [7] einen globalen Marktanteil im Desktop-Bereich von 3,64 %, was ihn nur mehr zum viertmeistgenutzten Browser im Desktopbereich macht.

1.4.5 Opera

*Opera*⁹ wird vom norwegischen Unternehmen Opera Software entwickelt, die erste Version erschien 1996. Bis 2005 war Opera kostenpflichtig bzw. mit Werbung versehen. Seit Version 8.50 ist der Browser vollständig kostenlos.

Bis Opera 12 wurde die eigens entwickelte Rendering-Engine *Presto* verwendet. Diese war bei der Umsetzung von Webstandards im Vordergrund und bot zusätzlich einen

⁷<https://www.apple.com/safari/>

⁸<https://www.firefox.com/>

⁹<https://www.opera.com/>

Interpretationsmodus für nicht standardkonformes (vor allem auf alte Internet Explorer optimiertes) HTML. Im Juli 2013 wurde ein Versionssprung auf Opera 15 vollzogen. Seit damals kommt – wie bei neuen Google Chrome und Microsoft Edge Versionen – die Blink Rendering-Engine zum Einsatz. Im April 2023 wurde Version 100 mit einem neuen User-Interface veröffentlicht. Die aktuelle Version ist 122.0. Opera weist laut [7] im Oktober einen Desktop-Marktanteil von 1,91 % auf.

1.5 HTML – Erste Schritte

Essentiell für einen erfolgreichen Start in die Webentwicklung ist das Verständnis ihrer zentralen Sprache – *HTML*. Beginnend mit diesem Abschnitt werden die wichtigsten Grundlagen dazu erläutert. Ziel ist es, ein Verständnis für den Aufbau und die Funktionsweise zu vermitteln, weniger jedes einzelne Element im Detail zu erklären. Für dies sei auf Dokumentationen im Web verwiesen – allen voran das Mozilla Developer Network¹⁰, welches eine gute Einführung inklusive aktueller Element-Referenz enthält.

1.5.1 Was ist HTML?

HTML – kurz für *Hypertext Markup Language* – ist die Beschreibungssprache für Web-Dokumente.

Hypertext bezeichnet, wie in Abschnitt 1.2.2 erwähnt, eine bestimmte Art von Text, die Informationen durch Hyperlinks mit anderen Hypertext-Knoten verknüpft.

Markup bedeutet zu Deutsch „auszeichnen“, also Textstellen zu kennzeichnen und ihnen somit eine spezielle Bedeutung zu geben. Diese Auszeichnung geschieht mit Hilfe von *HTML-Tags* (siehe dazu auch Abschnitt 1.5.2). Die ausgezeichneten Textpassagen werden vom Browser dann interpretiert und dargestellt (gerendert). Moderne Versionen von HTML zielen dabei auf eine semantische korrekte, d. h. logische Auszeichnung ab, die unmissverständlich bezeichnet, worum es sich bei einer Textstelle handelt (z. B. „Dieser Text ist eine Überschrift“). Details zu dieser Art von Auszeichnung finden sich in Vorlesung 2.

1.5.2 HTML-Tags und Elemente

HTML-Tags stehen immer in spitzen Klammern: `< >` – z. B. `<h1>`, `<p>`, `<a>` etc.

Der Großteil der HTML-Tags ist zweigeteilt, d. h. besitzt einen öffnenden und ein schließenden Tag. Der öffnende zeigt den Beginn der Auszeichnung an, es wird dazu der Name des Tags in spitze Klammern gestellt (z. B. `<h1>`). Der schließende Tag kennzeichnet das Ende der aktuellen Auszeichnung, es wird der Name des Tags hinter einen Schrägstrich (Slash) in die spitzen Klammern gestellt (z. B. `</h1>`). Um den Text „Das ist eine Überschrift“ als solche zu kennzeichnen, ist folgendes Markup notwendig:

```
1 <h1>Das ist eine Überschrift</h1>
```

Dieses Konstrukt aus Tags mitsamt Inhalt wird dann als *HTML-Element* bezeichnet.

¹⁰<https://developer.mozilla.org/en-US/docs/Web/HTML>

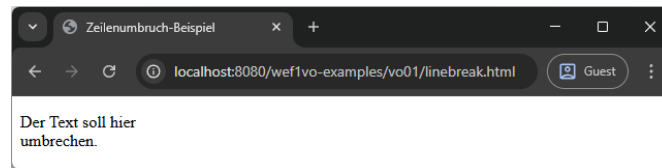


Abbildung 1.3: Der `<br>`-Tag bewirkt einen Zeilenumbruch nach dem Wort „hier“.

Case Sensitivity

Die Groß- und Kleinschreibung von HTML-Elementen kann prinzipiell variiert werden, d. h. es ist sowohl `<h1>` als auch `<H1>` für eine Überschrift erster Ordnung möglich. Es hat sich jedoch die klein geschriebene Variante in modernen Webseiten durchgesetzt.

Standalone-Tags

Während die meisten Elemente jeweils einen öffnenden und schließenden Teil besitzen, gibt es auch einige Tags, die nur aus einem Element (dem öffnenden Teil) bestehen. Sie werden *Standalone-Tags* (alleinstehende Tags) genannt und wirken in der Regel punktuell, d. h. sie kennzeichnen nicht einen Bereich wie andere HTML-Elemente. Der bekannteste Vertreter ist der Zeilenumbruch (engl. „line break“) `<br>`, der den Text innerhalb eines Absatzes an einer definierten Stelle umbricht. Um den Text „Der Text soll hier umbrechen.“ nach dem Wort „hier“ umzubrechen, d. h. das Wort „umbrechen“ in die nächste Zeile zu befördern, ist folgendes Markup notwendig:

```
1 <p>Der Text soll hier<br>umbrechen.</p>
```

Das Ergebnis ist in Abbildung 1.3 ersichtlich. Der Text selbst steht hier in einem Absatz (engl. „paragraph“), der durch das `<p>`-Element gekennzeichnet wird.

Wichtige HTML-Elemente für den Anfang

Tabelle 1.1 enthält zunächst eine Reihe von wichtigen Elementen für den Anfang. Mehr folgen in Vorlesung 2.

Verschachtelung

Viele (jedoch nicht alle) HTML-Elemente können wiederum andere HTML-Elemente enthalten. Man spricht in diesem Fall von *Verschachtelung*. Wichtig ist hierbei die korrekte Art und Weise, wie Tags ineinander verschachtelt werden. Überkreuzungen müssen unter allen Umständen vermieden werden. Als Regel gilt:

Werden mehrere HTML-Elemente ineinander verschachtelt, so sind sie genau in der umgekehrten Reihenfolge zu schließen, in der sie geöffnet wurden.

Korrekt ist also folgendes Markup:

```
1 <p><em>Dieses Markup ist korrekt verschachtelt.</em></p>
```

Inkorrekt wäre folgendes Beispiel:

```
1 <p><em>Achtung: Überkreuzung!</p></em>
```

Tabelle 1.1: Eine Liste der grundlegendsten HTML-Elemente.

Element	Bedeutung
<html>	Das sogenannte Root-Element. Jedes HTML-Dokument beginnt und endet mit ihm. Alle anderen Elemente sind in ihm enthalten.
<head>	In diesem Element sind alle Meta-Informationen zur aktuellen Seite enthalten. Zum Beispiel der Titel, Verweise auf Styles und Skripte oder der verwendete Zeichensatz.
<title>	Der Seitentitel. Er erscheint in der Regel in der Kopfzeile/im Tab des Browsers.
<body>	Innerhalb dieses Elements müssen alle Elemente vorkommen, die im eigentlichen Seiteninhalt aufscheinen sollen.
<h1>, <h2>, <h3>, <h4>, <h5>, <h6>	Sechs Ebenen von Überschriften. Sie sollten prinzipiell immer sequentiell verwendet werden (d. h. immer mit <h1> beginnen). Der Browser rendert sie von 1–6 in immer kleiner werdenden Größen, dies ist jedoch anpassbar (mehr dazu in Vorlesung 3).
<p>	Ein Absatz. Vor und nach ihm zeigt der Browser standardmäßig einen vertikalen Abstand an.
<div>	Ein Bereich bzw. Container. Dieses Element hat keine spezifische Bedeutung und wird oft für Layouts oder Gruppierung verwendet.
<a>	Ein Anker (engl. „anchor“) im Hypertext. Dieses Element ist der klassische Hyperlink.
 	Dieses Standalone-Element bewirkt einen Zeilenumbruch an der jeweiligen Stelle.
	Repräsentiert eine betonte Textstelle (engl. „emphasize“). Browser stellen diese Betonung standardmäßig kursiv dar.
	Bindet ein Bild bzw. eine Grafik ein.
	Ein generischer Bereich ähnlich <div>, jedoch auf der Textebene. Er kennzeichnet also nur einzelne Wörter.

Whitespaces im Quellcode

Gestaltlose Zeichen wie Leerzeichen, Tabulatoren oder auch Zeilenumbrüche im Quellcode (erzeugt durch Drücken der Enter-Taste, nicht durch Einfügen eines
-Tags) werden vom Browser prinzipiell ignoriert. Dies hat zur Folge, dass es keinen Unterschied für das Erscheinungsbild einer Webpage macht, ob der gesamte Code in einer oder jeder Zeile in seiner eigenen Zeile mit Einrückungen geschrieben wird.

So führt sowohl dieses Markup-Fragment aus drei aufeinanderfolgenden Absätzen

```
1 <p>Erster Absatz.</p>
2   <p>Zweiter Absatz.</p>
3 <p>Dritter Absatz.</p>
```

als auch dieses

```
1 <p>Erster Absatz.</p><p>Zweiter Absatz.</p><p>Dritter Absatz.</p>
```

zur Darstellung in Abbildung 1.4. Man beachte die Zeilenumbrüche, Einrückung und zusätzlichen Leerräume im ersten Code-Fragment. Diese fehlen im zweiten Fragment komplett und trotzdem ist das Ergebnis dasselbe, da der Browser diese gestaltlosen Zeichen ohnehin ignoriert. Verantwortlich für die Tatsache, dass im Browser die drei

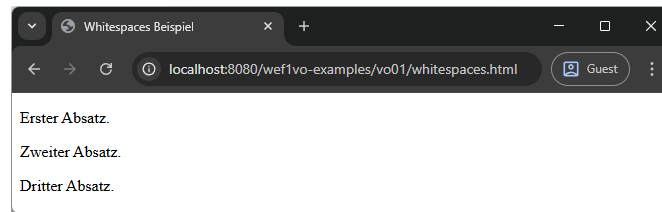


Abbildung 1.4: Egal ob Zeilenumbrüche, Einrückungen oder zusätzliche Leerräume verwendet werden – das Ergebnis im Browser sieht immer gleich aus: die drei Absätze stehen uneingerückt untereinander.

Zeilen untereinander stehen, ist das `<p>`-Element. Es erzeugt jeweils einen Absatz, der durch einen vertikalen Abstand vor und nach dem Text repräsentiert wird.

1.5.3 HTML-Attribute

Jeder HTML-Tag hat eine bestimmte Bedeutung, in vielen Fällen ist aber noch zusätzliche Information notwendig, um das Element etwa eindeutig zu kennzeichnen oder bei einem Hyperlink (repräsentiert durch den `<a>`-Tag) anzugeben, wohin der Link führt. Diese Zusatzinformationen können über *Attribute* angegeben werden. Ein Element kann ein oder mehrere Attribute besitzen, diese werden in der Form `name="wert"` in den öffnenden HTML-Tag geschrieben. Um etwa einem Absatz die eindeutige Bezeichnung (ID) `content` zu geben, ist folgendes Markup nötig:

```
1 <p id="content">Etwas Inhalt</p>
```

Jeder Tag hat eine unterschiedliche Menge an möglichen Attributen, diese sind in der Dokumentation des HTML Living Standard [8] definiert. Es gibt jedoch eine Reihe von sogenannten globalen Attributen, die bei jedem Element angefügt werden können. `id` ist ein derartiges globales Attribut. Eine vollständige Auflistung für die aktuellste HTML-Version findet sich in der Living Standard-Spezifikation der WHATWG¹¹ oder beim Mozilla Developer Network¹².

1.5.4 Kommentare

Kommentare im Code dienen dazu, Zusatzinformationen anzumerken, die jedoch nicht in der Webseite selbst ersichtlich sein sollen. So können etwa am Beginn des Dokuments Autor*innenname, Erstellungsdatum oder Versionsnummer angegeben werden. Im Code können bei komplexeren Stellen Anmerkungen gemacht werden. Der Browser ignoriert diese Teile vollständig und zeigt den Inhalt nicht an. Da aber Kommentare Teil des Quelltextes sind, sind sie von jeder Person, die sich den Quellcode der Webseite im Browser ansieht, lesbar.

Kommentare können weiters dazu dienen, ganze Blöcke im HTML-Markup auszukommentieren, z. B. um Fehler zu suchen, oder nicht benötigte Teile temporär zu entfernen.

¹¹<https://html.spec.whatwg.org/multipage/dom.html#global-attributes>

¹²https://developer.mozilla.org/en-US/docs/Web/HTML/Global_attributes

Ein HTML-Kommentar beginnt mit der öffnenden spitzen Klammer, gefolgt von einem Ausrufezeichen und zwei Bindestrichen. Um das Kommentar zu schließen sind zwei Bindestriche gefolgt von der schließenden spitzen Klammer nötig. Ein einzeiliges HTML-Kommentar sieht folgendermaßen aus:

```
1 <!-- <p>Dieser Absatz wird vom Browser nicht angezeigt.</p> -->
```

Kommentare können sich auch über mehrere Zeilen erstrecken bzw. auch mitten im Markup vorkommen:

```
1 <!--
2 <p>Ein Absatz.</p>
3 <p>Noch ein Absatz. Beide werden auskommentiert.</p>
4 -->
5 <p>Dieser Teil ist <!-- nicht sichtbar --></p>
```

1.5.5 Entitäten

Einige Zeichen sind in HTML für interne Funktionalitäten in Verwendung. So werden die spitzen Klammern `<` und `>` verwendet, um einen HTML-Tag anzuzeigen. Was nun, wenn im Text eines dieser Zeichen benötigt wird? Würde es einfach so verwendet werden, ginge der Browser davon aus, dass ein HTML-Tag beginnt und könnte evtl. unlesbaren Output erzeugen.

Um dies zu verhindern, können die spitzen Klammern speziell kodiert werden. So wird dem Browser mitgeteilt, dass explizit das Zeichen angezeigt werden soll und nicht ein HTML-Tag beginnt. Diese Kodierungen werden *Entitäten* (engl. „entities“) genannt. Sie beginnen immer mit einem kaufmännischen Und (`&`) und enden mit einem Strichpunkt (`;`). Dazwischen kommt der jeweilige Bezeichner.

Um nun die öffnende spitze Klammer (auch „Kleiner-Zeichen“) zu erhalten, lautet die Entität `<` (lt – lower than). Die schließende spitze Klammer wird durch `>` (gt – greater than) repräsentiert. Auch für deutsche Umlaute und viele Sonderzeichen gibt es Entitäten. Diese mussten früher verwendet werden, als hauptsächlich noch regionale Zeichenkodierungen im Einsatz waren. Seit der starken Verbreitung von *Unicode* (und den damit verbundenen Kodierungen) sind diese nicht mehr zwingend nötig (mehr zu Zeichenkodierungen in Vorlesung 2). Jedoch ist es – gerade bei Sonderzeichen – oft schneller, den Entitätsnamen zu verwenden, als das Sonderzeichen über Copy & Paste einzufügen (z. B. bei `©`, um ein ©-Zeichen zu erzeugen).

Tabelle 1.2 enthält eine kleine Übersicht über einige Entitäten. Eine vollständige Liste findet sich in im HTML Living Standard¹³.

1.5.6 HTML-Grundgerüst

Mit dem Wissen über Elemente und korrekte Verschachtelung kann nun ein erstes, einfaches HTML-Grundgerüst aufstellt werden. Es gibt der Seite einen Titel und zeigt einen einzelnen Absatz mit etwas Inhalt an. Zur besseren Übersicht wurden die Elemente entsprechend ihrer Verschachtelungstiefe eingerückt. Die Ausgabe des Grundgerüsts ist in Abbildung 1.5 ersichtlich.

¹³<https://html.spec.whatwg.org/multipage/named-characters.html#named-character-references>

Tabelle 1.2: Eine Liste mit einigen wichtigen HTML-Entitäten. ` ` steht dabei für ein geschütztes Leerzeichen (engl. „non-breaking space“).

Zeichen	Entität	Zeichen	Entität
<	<code>&lt;</code>	ä	<code>&auml;</code>
>	<code>&gt;</code>	ö	<code>&ouml;</code>
&	<code>&amp;</code>	ü	<code>&uuml;</code>
"	<code>&quot;</code>	Ä	<code>&Auml;</code>
'	<code>&apos;</code>	Ö	<code>&Ouml;</code>
©	<code>&copy;</code>	Ü	<code>&Uuml;</code>
□	<code>&nbsp;</code>	ß	<code>&szlig;</code>

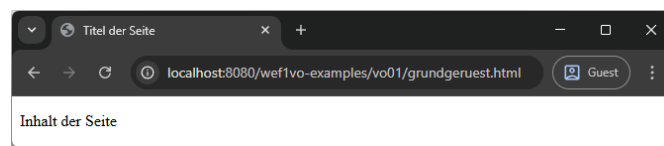


Abbildung 1.5: Das einfachste HTML-Grundgerüst im Browser dargestellt.

```

1 <!DOCTYPE html>
2 <html lang="de">
3   <head>
4     <title>Titel der Seite</title>
5   </head>
6   <body>
7     <p>Inhalt der Seite</p>
8   </body>
9 </html>

```

1.5.7 Der DOCTYPE

Der *DOCTYPE* spezifiziert die verwendete HTML-Version den am Beginn eines HTML-Dokuments. Er garantiert, dass die Seite vom Browser nach den Regeln dieser Version interpretiert wird. DOCTYPEs wurden zum ersten Mal um die Jahrtausendwende mit der Einführung von HTML 4.01 bzw. des Browsers Internet Explorer 5 für macOS notwendig (die ersten DOCTYPEs kamen aber bereits zwei Jahre vorher mit der Veröffentlichung von HTML 4.0 auf). Dieser Browser hatte eine komplett neue Rendering-Engine (genannt *Tasman*), welche sich – verglichen zur parallel entwickelten und verwendeten Trident Engine – viel mehr an Standards (wie das neue HTML 4.01) hielt. Dies hatte zur Folge, dass alte Websites nicht mehr korrekt angezeigt wurden. Mark Pilgrim dazu in [6]:

IE5/Mac was so advanced, it actually broke the Web.

Microsofts Lösung war, den sogenannten DOCTYPE, als erste Zeile eines Dokuments, noch vor dem `<html>`-Element auszulesen. War ein solcher vorhanden, wurde das Dokument standardkonform gerendert, wenn nicht, dann wurde die alte (bisher verwendete)

Methode verwendet. Dieser Modus trägt inzwischen den Namen *Quirks-Modus*, sein Gegenstück ist (wie erwartet) der *Standard-Modus*.

Diese Lösung ermöglicht, dass Browser Markup, das nach einer bestimmten HTML-Version geschrieben wurde, korrekt ausgegeben können. Auch dann noch, wenn sich der Standard inzwischen stark geändert haben sollte.

Für HTML 4.01 gab es drei DOCTYPEs. Der DOCTYPE für den strikten HTML 4.01 Standard (HTML 4.01 Strict), der für den etwas lockeren HTML 4.01 Transitional Standard und ein Standard für Websites mit Frames (HTML 4.01 Frameset). Diese DOCTYPEs sind nach einer eigenen Grammatik aufgebaut, hinter dem URL steckt die Definition, welche Elemente genau wie verwendet werden dürfen:

```
1 <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN"
2   "http://www.w3.org/TR/html4/strict.dtd">
```

```
1 <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
2   "http://www.w3.org/TR/html4/loose.dtd">
```

```
1 <!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Frameset//EN"
2   "http://www.w3.org/TR/html4/frameset.dtd">
```

Für XHTML 1.0, den Nachfolger von HTML 4.01, gab es ebenfalls eigene DOCTYPES für dieselben drei Varianten:

```
1 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
2   "http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
```

```
1 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
2   "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
```

```
1 <!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Frameset//EN"
2   "http://www.w3.org/TR/xhtml1/DTD/xhtml1-frameset.dtd">
```

Während diese Zeilen ihren Zweck erfüllen, sind sie für Entwickler*innen in der Regel nicht auswendig merkbar. Mit HTML5 (nun mehr der HTML Living Standard) wurde auf diese Tatsache reagiert, der DOCTYPE für diese Version sieht (wie bereits im Grundgerüst verwendet) wie folgt aus. Unterschiedliche Substandards wie Strict oder Transitional gibt es nicht mehr. Auch neue Entwicklungen, wie sie laufend im Living Standard geschehen, werden diese Syntax beibehalten.

```
1 <!DOCTYPE html>
```

1.6 Die Geschichte von HTML

Die Geburtsstunde von *HTML* war 1989 im Schweizer Forschungsinstitut CERN. Tim Berners-Lee entwickelte dort den Prototypen eines Datenformats, das es erlaubte, Dokumente untereinander zu referenzieren. Während auch Firmen wie Apple an Hypertext-Systemen arbeiteten, diese aber nur auf lokale Arbeit an einem Computer ausgelegt waren, sah die Idee von Berners-Lee vor, ein globales System zu schaffen, das über das Internet funktionieren sollte.

Dazu wurde eine einfache Art benötigt, diese Daten zu übertragen und Berners-Lee entwickelte hierfür das HTTP. Das Format für Dokumente wurde HTML, hier

griff Berners-Lee jedoch auf die damals bereits bekannte und auch akzeptierte Markup-Sprache SGML (Standard Generalized Markup Language) zurück und basierte HTML zu großen Teilen darauf. Dies erwies sich als kluger Schachzug und ebnete den Weg für den Siegeszug von HTML.

Im Folgenden sind einige der wichtigsten Schritte in der weiteren Geschichte von HTML aufgelistet:

- **1992:** Die Urversion von *HTML* wird in einem Dokument spezifiziert. Sie kann zunächst nur mit Text arbeiten. Später kommen noch einige einfache Formatierungen und das Einbinden von Bildern hinzu.
- **1993:** Mit *HTML+* werden Erweiterungen wie der Textumfluss von Bildern vorgeschlagen, jedoch nie wirklich verabschiedet.
- **1995:** *HTML 2.0* ist die erste, über einen sog. Request for Comments (RFC) definierte Version. Neu sind etwa Formulare.
- **1997:** Anfang des Jahres bringt *HTML 3.2* Tabellen und nun auch den Textfluss um Bilder. Java Applets kommen auch zum ersten Mal auf. Die zuvor geplante Version 3.0 wird gleich übersprungen.
- **1997:** Ende des Jahres erscheint *HTML 4.0*. Zu den Neuigkeiten zählen Stylesheets, Frames, Scripts und viele anderen Dinge. Zum ersten Mal wird von Strict, Transitional und Frameset Varianten gesprochen.
- **1999:** Viele kleine Fehler in HTML 4.0 führen zur Einführung der *Version 4.01*. Sie bleibt die letzte offizielle HTML-Version bis HTML5.
- **2000:** *XHTML 1.0* wird als Neuformulierung von HTML 4.01 in der Markupsprache XML vorgestellt. Die Version wird 2002 noch einmal überarbeitet.
- **2001:** *XHTML 1.1* setzt den Kurs seines Vorgängers fort. Die Sprache wird in Module aufgeteilt. Nur die strikte Variante bleibt übrig. XHTML 1.1 findet jedoch weniger Verbreitung als sein Vorgänger.
- **2006:** Mit *XHTML 2.0* sollte eine radikale Neuformulierung der Sprache realisiert werden. Aufgrund der strikteren Syntax zog die lediglich als Entwurf vorliegende Spezifikation 2009 den kürzeren gegenüber dem parallel entwickelten HTML5 und die Entwicklung wurde eingestellt.
- **2009:** Der erste Entwurf der neuen *HTML5* Spezifikation wird vorgelegt. Er wird ständig weiter entwickelt und stellt derzeit den Stand der Technik dar. Aufbauend auf den Vorgänger (HTML 4.01) werden neues Vokabular eingeführt und Altlasten entsorgt.
- **2012:** Noch bevor HTML5 als Recommendation verabschiedet ist, wird Ende des Jahres der erste Entwurf von *HTML 5.1* veröffentlicht. Er soll neue Tags und Funktionen einführen, die in Version 5 keinen Platz mehr finden.
- **2014:** *HTML5* wird am 28. Oktober 2014 zum Standard (W3C Recommendation) erhoben.
- **2016:** Am 1. November wird *HTML 5.1* als Recommendation verabschiedet. Die Entwicklungszeit betrug knappe 4 Jahre. Weiters wird ein erster Entwurf für HTML 5.2 im August 2016 veröffentlicht. Er umfasst vor allem Korrekturen für Version 5.1.

- **2017:** *HTML 5.2* wird am 14. Dezember zur Recommendation erhoben und stellt somit den letztgültigen Standard da. Zum selben Zeitpunkt wird mit der Arbeit an HTML 5.3 begonnen.
- **2018:** HTML 5.3 steht seit Oktober 2018 im Status eines Working Draft, also noch am Anfang des Standardisierungsprozesses. Der angepeilte, jährliche Veröffentlichungszyklus wurde nicht mehr eingehalten, parallele Entwicklungen der WHATWG mit dem Living Standard führten zu Verzögerungen.
- **2019:** Am 28. Mai beschließen W3C und WHATWG (siehe Abschnitt 1.7) die Entwicklung von HTML auf den WHATWG *HTML Living Standard* zu verlegen. Dieser stellt somit immer den letztgültigen Entwicklungsstand dar. Die Versionierungen von HTML5 gehören auch der Vergangenheit an. Entsprechende (veraltete) Standards leiten mittlerweile auf den Living Standard weiter.

1.7 Standardisierung im Web

Damit die eingesetzten Technologien im Web nicht einem Wildwuchs durch im Wettbewerb liegende Browserhersteller ausgesetzt sind (wie es schon in den 90ern zwischen Microsoft und Netscape der Fall war) gibt es *Standards*, an die sich Browserhersteller halten können (jedoch nicht müssen). Diese Standards sollen sicherstellen, dass das Erlebnis einer Website unabhängig vom Client dasselbe ist bzw. Benutzer*innen nicht gezwungen sind, für eine bestimmte Seite einen bestimmten Browser zu verwenden.

1.7.1 Organisationen

Zwei wichtige Organisationen sind führend bei der Entwicklung von Standards im Web.

Das W3C

Das Unternehmen, das diese Standardisierungen vornimmt, ist das *World Wide Web Consortium*¹⁴ (kurz W3C). Es wurde 1994 von Tim Berners-Lee gegründet. Es überwacht und entwickelt eine Vielzahl an Webtechnologien und -funktionen, allen voran CSS, aber auch XML, Barrierefrei usw.

Jede neue Technologie-Spezifikation für das Web wird jedoch nicht sofort zum Standard erklärt. Sie durchläuft eine jahrelange Phase permanenter Überarbeitung und Reevaluierung bis sie als Standard verabschiedet wird.

Die Stufen dieses Prozesses sind folgende:

1. Working Draft (WD) – Arbeitsentwurf,
2. Last Call (LC) – Letzter Aufruf für Kommentare zum Entwurf,
3. Candidate Recommendation (CR) – Veröffentlichungsvorschlag,
4. Proposed Recommendation (PR) – Geplante Veröffentlichung,
5. W3C Candidate Recommendation (REC) – Standard.

Da das W3C kein klassisches Standardisierungsunternehmen (wie etwa die ISO) ist, spricht man genau genommen auch nicht von einem Standard, sondern von einer Empfehlung. De facto handelt es sich aber um einen anerkannten Standard.

¹⁴<https://www.w3.org/>

Die WHATWG

Während das W3C ein unabhängiges Konsortium ist, dem vornehmlich Einzelpersonen angehören, hat sich parallel dazu eine Arbeitsgruppe, bestehend aus Personen, die sehr unternehmensnah agierten, gebildet. Zur *Web Hypertext Application Technology Working Group*¹⁵ (WHATWG) gehören etwa Personen der Mozilla Foundation, Opera Software AS, Google oder Apple. Sie wurde gegründet, da das W3C was die Entwicklung von Standards angeht, von vielen (vor allem den Browser Herstellern) sehr lange als zu träge angesehen wurde. Während viele Jahre so etwas wie direkte Konkurrenz herrschte, die in den Parallelentwicklungen XHTML 2.0 (W3C) und HTML5 (WHATWG) gipfelte, gibt es in den letzten Jahren wieder aktive Zusammenarbeit.

Die WHATWG strebt keine Konkurrenz zum W3C im Standardisierungsprozess an. Vielmehr werden WHATWG-Entwürfe, die sogenannten *Living Standards*, vom W3C mitentwickelt. Das bekannteste Beispiel ist der HTML Living Standard [8].

1.7.2 Überprüfen von Standards

Das W3C stellt Entwickler*innen Tools zur Verfügung, die es ermöglichen, die eigene Website auf Standardkonformität zu prüfen. Im Falle von HTML ist dies der *HTML-Validator*¹⁶. Der Validator analysiert das Markup der eigenen Seite und kann aufgrund des verwendeten DOCTYPEs feststellen, ob der angegebene Standard eingehalten wurde. Mögliche Fehler werden mit Erklärungen ausgegeben.

Der Validator funktioniert dem aktuellen HTML Living Standard, aber auch ältere Versionen wie HTML 4.01 oder XHTML 1.0 können überprüft werden.

1.8 Weiterführende Literatur

Wie bereits erwähnt, befindet sich HTML in stetiger Weiterentwicklung. Mit dem Kauf von Literatur passiert es daher unausweichlich, dass diese unter Umständen nach kurzer Zeit bereits nicht mehr aktuell ist. Davon sollte man sich jedoch nicht abschrecken lassen, sind die Änderungen, die laufend im Living Standard geschehen, selten so groß, dass Bücher völlig unbrauchbar werden. Dennoch sollte man bei Literatur darauf achten, dass sie nicht älter als ein paar Jahre ist.

Wer HTML zusammen mit CSS abdecken möchte, kann auf [11] zurückgreifen. Das Buch ist seit Jahren mit kontinuierlichen Neuauflagen das aktuellste und orientiert sich am Living Standard. Zusammen mit den Themen CSS und JavaScript werden viele Dinge der Hypermedia-Vorlesung auf über 1.200 Seiten abgedeckt.

Drei weitere, zwischen 400 und 500 Seiten starke, Einführungswerke sind [4], [10] und [3]. Alle konzentrieren sich auf die Themen HTML und CSS (also kein JavaScript) und legen den Aufbau einer ersten, einfachen Webseite ins Zentrum.

¹⁵<https://whatwg.org/>

¹⁶<https://validator.w3.org/>

Quellenverzeichnis

- [1] Tim Berners-Lee, Roy Fielding und Larry Masinter. *Uniform Resource Identifier (URI): Generic Syntax*. RFC 3986. IETF, Jan. 2005, S. 1–61. URL: <https://datatracker.ietf.org/doc/rfc3986/> (siehe S. 1-3).
- [2] Tim Berners-Lee, Larry Masinter und Mark McCahill. *Uniform Resource Locators (URL)*. RFC 1738. IETF, Dez. 1994, S. 1–25. URL: <https://datatracker.ietf.org/doc/rfc1738/> (siehe S. 1-3).
- [3] Paul Fuchs. *HTML5 und CSS3 für Einsteiger. Der leichte Weg zur eigenen Webseite*. Landshut, Deutschland: BMU Verlag, 21. Juni 2019 (siehe S. 1-16).
- [4] Peter Müller. *Einstieg in HTML und CSS. Webseiten programmieren und gestalten*. 3. Aufl. Bonn, Deutschland: Rheinwerk Computing, 3. Sep. 2024 (siehe S. 1-16).
- [5] Theodore H. Nelson. *Computer Lib/Dream Machines*. 2. Aufl. Tempus Books of Microsoft Press, 1. Okt. 1987 (siehe S. 1-2).
- [6] Mark Pilgrim. *HTML5. Up and Running*. Sebastopol, USA: O'Reilly, 6. Aug. 2010 (siehe S. 1-12).
- [7] StatCounter. *Desktop Browser Market Share Worldwide*. Okt. 2025. Okt. 2025. URL: <https://gs.statcounter.com/browser-market-share/desktop/worldwide/#monthly-202510-202510-bar> (besucht am 21.10.2025) (siehe S. 1-5, 1-6, 1-7).
- [8] Web Hypertext Application Technology Working Group. *HTML*. Living Standard. 20. Okt. 2025. URL: <https://html.spec.whatwg.org/multipage/> (siehe S. 1-10, 1-16).
- [9] Web Hypertext Application Technology Working Group. *URL*. Living Standard. 18. Aug. 2025. URL: <https://url.spec.whatwg.org/> (siehe S. 1-3).
- [10] Christian Wenz und Christoph Prevezanos. *Jetzt lerne ich HTML5 und CSS3*. 2. Aufl. Burgthann, Deutschland: Markt + Technik Verlag, 16. Nov. 2022 (siehe S. 1-16).
- [11] Jürgen Wolf. *HTML und CSS. Das umfassende Handbuch*. 5. Aufl. Bonn, Deutschland: Rheinwerk Computing, 4. Aug. 2023 (siehe S. 1-16).